



Ministère de l'Enseignement Supérieur

et

De la Recherche Scientifique

Institut Supérieur

De Technologie



IIT
INSTITUT
INTERNATIONAL
TECHNOLOGIE

وزارة التعليم العالي والبحث العلمي
المدرسة العليا الدولية الخاصة
للتكنولوجيا
بصفاقس

Rapport de Stage de Fin d'études

Effectué à: Logicom, Sfax



Élaboré par :

Ben Moussa Zied

Hamden Med Taher

En vue de l'obtention du Diplôme National de Licence (3 ans) en

Informatique

Intitulé :

**Espace Coworking et
Formation**

Ahmed Fakhfekh : Directeur de thèse

Wafa Gouiaa : Superviseure industrielle

Tasnim Sakka : Rapporteur

Année Universitaire : 2025-2026

Ministère de l'Enseignement Supérieur
et
De la Recherche Scientifique
Institut Supérieur
De Technologie

وزارة التعليم العالي والبحث العلمي
المدرسة العليا الدولية الخاصة
للتكنولوجيا
بصفاقس

Rapport de Stage de Fin d'études

Effectué à: Logicom, Sfax



Élaboré par :

Ben Moussa Zied

Hamden Med Taher

En vue de l'obtention du Diplôme National de Licence (3 ans) en

Informatique

Intitulé :

**Espace Coworking et
Formation**

Ahmed Fakhfekh : Directeur de thèse

Wafa Gouiaa : Superviseure industrielle

Tasnim Sakka : Rapporteur

Année Universitaire : 2025-2026

Dédicaces

C'est avec une profonde gratitude que nous dédions ce modeste travail

Dédicace de Ben Moussa Zied

À mon cher père (**Abdelkader Ben Moussa**)

Je voulais te dire combien je t'aime et à quel point je suis reconnaissant pour tout ce que tu as fait pour moi. Tu es mon héros, celui qui m'inspire chaque jour à devenir une meilleure personne. Merci pour ton amour inconditionnel, ta sagesse et ton soutien constant.

À l'âme de ma chère mère (**Saida Souissi**)

Aucune parole ne pourra exprimer pleinement l'amour, le manque et la gratitude que je porte dans mon cœur. Tu as été ma force dans les moments difficiles, mon inspiration dans les instants de doute et la lumière qui a guidé chacun de mes pas.

Aujourd'hui, alors que j'achève ce travail, j'aurais tant voulu partager cette fierté avec toi. Même si tu n'es plus parmi nous, ta présence demeure vivante dans mon cœur, dans mes pensées et dans chacune de mes réussites.

Ce rapport de fin d'études t'est dédié, en témoignage de tout l'amour, des sacrifices et des valeurs que tu m'as transmis. J'espère être à la hauteur des rêves que tu portais pour moi.

Que Dieu t'accorde Sa miséricorde et t'accueille dans Son vaste paradis.

À toi, ma mère bien-aimée, repose en paix.

À ma Chère tante (**Halima Souissi**)

Vous Les mots ne suffisent pas pour exprimer le vide laissé par ton absence, ni la place immense que tu occupes dans mon cœur. Ton affection, ta bienveillance et tes encouragements resteront à jamais gravés dans ma mémoire.

En ce jour où j'achève ce travail, j'aurais aimé partager ce moment avec toi. Même si tu n'es plus parmi nous, ton souvenir continue de m'accompagner et de m'inspirer chaque jour.

Je te dédie humblement ce rapport, en témoignage de mon amour, de mon respect et de ma profonde gratitude pour tout ce que tu as représenté dans ma vie.

Que Dieu t'accorde Sa miséricorde infinie et t'accueille dans Son vaste paradis.

Repose en paix, chère tante. Tu resteras à jamais dans mon cœur.

À mes chers frères, **Tarek, Firas, Bassem et Mahdi,**

Avec toute mon affection et ma profonde gratitude, je vous dédie ce travail en témoignage de l'amour fraternel, du soutien et des encouragements que vous m'avez apportés tout au long de mon parcours. Merci d'avoir toujours été présents, dans les moments de doute comme dans les instants de joie. Votre présence, vos conseils et votre confiance en moi ont été une source de motivation et de force. Que cette réussite soit aussi la vôtre, car votre soutien a contribué, de près ou de loin, à l'accomplissement de ce travail.

Avec tout mon amour et mon respect.

À mon encadrant professionnel Mr Ahmed Fakhfekh

Je tiens à exprimer ma profonde gratitude et mes sincères remerciements à mon encadrant professionnel, **Monsieur Ahmed Fakhfekh**, pour son accompagnement précieux tout au long de ce projet. Je le remercie vivement pour sa disponibilité, ses conseils pertinents, ainsi que pour le temps qu'il m'a accordé afin de m'orienter et de m'aider à mener à bien ce travail. Son encadrement, ses remarques constructives et son soutien constant ont été d'une grande importance dans la réalisation de ce projet. Veuillez trouver ici l'expression de mon profond respect et de ma reconnaissance.

Dédicace De Med Taher Hamden

À mon cher père (Mohamed Raouf Hamden)

Avec toute ma gratitude, mon respect et mon affection, je te dédie ce travail. Merci pour ton soutien inconditionnel, tes sacrifices, tes conseils précieux et la confiance que tu as toujours placée en moi. Ta présence et tes encouragements ont été une source de motivation tout au long de mon parcours.

À mon chère mère (Ghazela Ben Salem)

Je dédie ce travail à la personne qui a toujours été à mes côtés, m'offrant son amour, son soutien et ses encouragements sans limite. Merci pour tous les sacrifices que tu as consentis pour mon bonheur et ma réussite. Ta patience, ta tendresse et ta confiance en moi ont été une source de force et de motivation tout au long de mon parcours. Aucune parole ne saurait exprimer toute ma gratitude et mon affection

envers toi.

À mes chers frères (Khaled, Marwa, Safa et Ayoub)

Je dédie ce travail avec beaucoup d'affection et de gratitude. Merci pour votre soutien, votre présence et vos encouragements tout au long de mon parcours. Votre aide, vos conseils et votre confiance en moi ont été une source de motivation importante. Je vous remercie pour tous les moments de solidarité et d'amour fraternel que nous partageons. Que ce travail soit le témoignage de mon profond attachement envers vous. À mes amies, Je dédie ce travail avec beaucoup d'affection et de reconnaissance. Merci pour votre amitié sincère, votre soutien et votre présence à mes côtés dans les moments faciles comme dans les moments difficiles. Votre encouragement et votre bonne humeur ont été pour moi une véritable source de motivation tout au long de mon parcours. Que ce travail soit le symbole de notre belle amitié.

À mon encadrant professionnel (Mr. Ahmed Fakhfekh)

Je tiens à exprimer ma profonde gratitude et mes sincères remerciements à mon encadrant professionnel, Monsieur Ahmed Fakhfekh, pour son accompagnement précieux tout au long de ce projet. Je le remercie vivement pour sa disponibilité, ses conseils pertinents, ainsi que pour le temps qu'il m'a accordé afin de m'orienter et de m'aider à mener à bien ce travail. Son encadrement, ses remarques constructives et son soutien constant ont été d'une grande importance dans la réalisation de ce projet. Veuillez trouver ici l'expression de mon profond respect et de ma reconnaissance

Sommaire

Introduction Générale

Chapitre 1 : Présentation de la société d'accueil et contexte du stage

- 1.1 Introduction
- 1.2 Présentation de la société Logicom
- 1.3 Secteurs d'activités
- 1.4 Présentation du projet
- 1.5 Présentation de la méthode agile « Scrum »
- 1.6 Conclusion

Chapitre 2 : Analyse et spécification des besoins

- 2.1 Introduction
- 2.2 Analyse de l'existant et capture des besoins
- 2.3 Identification des acteurs
- 2.4 Spécifications fonctionnelles
- 2.5 Besoins non-fonctionnels
- 2.6 Conclusion

Chapitre 3 : Conception de la solution

- 3.1 Architecture technique
- 3.2 Conception de la base de données et schéma relationnel
- 3.3 Logique métier et relations inter-modules
- 3.4 Conclusion

Chapitre 4 : Réalisation technique

- 4.1 Environnement de développement et stack technologique
- 4.2 Développement du Frontend (Single Page Application)
- 4.3 Interface d'administration (Dashboard)
- 4.4 Implémentation des flux critiques
- 4.5 Conclusion

Chapitre 5 : Documentation et tests des API

5.1 Introduction

5.2 Outils et environnement de test

5.3 Documentation des endpoints de l'API

5.4 Tests fonctionnels illustrés

5.5 Sécurité de l'API

5.6 Conclusion

Conclusion Générale

Introduction Générale

Le présent rapport expose les travaux de conception et de développement de la plateforme TZ Prime Solutions, réalisés dans le cadre de notre projet de fin d'études au sein de la société Logicom, à Sfax. Ce projet est né d'une vision stratégique visant à fusionner deux univers complémentaires : l'espace de travail collaboratif et l'apprentissage professionnel.

Dans un contexte marqué par une transformation digitale accélérée, les besoins en matière d'accès rapide, simple et centralisé aux services de coworking et de formation n'ont cessé de croître. Les solutions existantes sur le marché traitent généralement ces deux besoins de manière séparée, ce qui limite l'efficacité et la fluidité des opérations pour les utilisateurs comme pour les gestionnaires. C'est dans cette optique que s'inscrit notre projet : concevoir et développer une plateforme web unifiée capable de centraliser la gestion des espaces de coworking et des programmes de formation au sein d'un même écosystème numérique.

L'objectif majeur de TZ Prime Solutions est ainsi de centraliser, au sein d'un environnement digital unique, les services de coworking et les programmes de formation. Cette double vocation constitue la proposition de valeur unique (UVP) de la plateforme : offrir aux utilisateurs un environnement hybride où ils peuvent travailler et apprendre simultanément. En automatisant la gestion administrative et en modernisant l'accès aux ressources grâce aux technologies React et Node.js, TZ Prime Solutions se positionne comme un levier de productivité et de montée en compétences pour ses utilisateurs.

Pour mener à bien ce travail, nous avons structuré ce rapport en cinq chapitres. Le premier chapitre présente la société d'accueil Logicom ainsi que le contexte et la problématique ayant motivé ce projet. Le deuxième chapitre est consacré à l'analyse et à la spécification des besoins fonctionnels et non-fonctionnels de la plateforme. Le troisième chapitre détaille la conception de la solution, notamment l'architecture technique adoptée et la modélisation de la base de données. Le quatrième chapitre décrit la réalisation technique de la plateforme, à travers la présentation de l'environnement de développement, des choix technologiques et de l'implémentation des principales fonctionnalités. Enfin, le cinquième chapitre présente la documentation des endpoints de l'API REST développée ainsi que leur validation à travers des tests fonctionnels.

Chapitre 1

Présentation de la société d'accueil et contexte du stage

1.1 Introduction

Avant d'aborder l'étude détaillée de notre projet de fin d'études, nous présenterons brièvement, dans ce premier chapitre, l'organisme d'accueil au sein duquel s'inscrit notre travail, à savoir la société Logicom. Nous poursuivrons par une présentation du contexte et de la problématique ayant motivé ce projet, puis par une analyse de l'existant mettant en évidence les limites des solutions actuelles sur le marché du coworking et de la formation en ligne. Cette analyse nous amènera à proposer une solution numérique adaptée sous forme d'une plateforme web unifiée, TZ Prime Solutions. Nous clôturerons ce chapitre par une présentation de la méthodologie agile Scrum, adoptée tout au long du développement de notre solution.

1.2 Présentation de la société Logicom

L'entreprise LOGICOM Informatique Sfax est une société tunisienne spécialisée dans le développement de logiciels de gestion, de solutions ERP et d'applications web destinées aux entreprises de différents secteurs d'activité. Fondée en 1998, elle accompagne les organisations dans leur transformation digitale à travers la conception et l'intégration de solutions informatiques adaptées à leurs besoins.

Grâce à son expertise technique et à son expérience dans le domaine des systèmes d'information, LOGICOM propose des solutions permettant d'améliorer la productivité, la rentabilité et la gestion des processus métiers des entreprises.



Figure 1.1. Logo de la société Logicom

1.3 Secteurs d'activités

Le tableau 1.1 présente les principaux secteurs d'activité dans lesquels intervient la société Logicom.

Secteur d'activité		Description
Développement	de	Conception et développement de solutions informatiques

Secteur d'activité	Description
logiciels	adaptées aux besoins des entreprises.
Solutions ERP	Mise en place de systèmes intégrés permettant la gestion globale des activités de l'entreprise.
Applications Web	Développement d'applications web pour la gestion et l'automatisation des processus métiers.
Gestion commerciale	Solutions pour la gestion des ventes, achats, stocks, clients et fournisseurs.
Comptabilité et finance	Développement d'outils de gestion comptable, financière et fiscale.
Gestion des ressources humaines	Solutions dédiées à la gestion du personnel, de la paie, des congés et des carrières.
Gestion de la restauration	Systèmes de gestion des restaurants, points de vente et caisses.

Table 1.1 – Secteurs d'activités de Logicom

1.4 Présentation du projet

Nous présenterons dans cette partie le cadre de notre projet, la problématique, l'étude de l'existant et la solution proposée.

1.4.1 Cadre du projet

Le présent travail s'inscrit dans le cadre du projet de fin d'études en vue de l'obtention du Diplôme National de Licence en Informatique, à l'École Supérieure Internationale Privée de Technologie de Sfax (IIT). Ce projet, réalisé au sein de la société Logicom, consiste à développer une plateforme web baptisée **TZ Prime Solutions**, dédiée à la gestion d'un espace de coworking et de programmes de formation. Cette double vocation — travailler et apprendre — constitue la proposition de valeur unique (UVP) de la plateforme, qui vise à centraliser, au sein d'un écosystème digital unique, les services de réservation d'espaces de travail collaboratif et les inscriptions aux formations professionnelles.

1.4.2 Problématique

Dans un contexte de transformation digitale, Logicom souhaite moderniser la gestion de ses services liés à la formation et au coworking. Actuellement, la gestion de ces activités repose sur des méthodes classiques ou

des outils dispersés, ce qui limite l'efficacité et la fluidité des opérations. La problématique principale réside dans la difficulté de gérer de manière efficace et unifiée les formations, les réservations des espaces de coworking ainsi que les inscriptions des utilisateurs. L'absence d'un système centralisé peut entraîner des pertes de temps, un manque de coordination, des erreurs de gestion et une mauvaise visibilité des disponibilités.

Plusieurs questions se posent alors :

- Comment concevoir une interface unique permettant de gérer à la fois les réservations d'espaces de coworking (salles, emplacements, tables) et les inscriptions aux formations ?
- Quelles fonctionnalités clés intégrer pour répondre aux besoins spécifiques des utilisateurs (administrateurs et membres) ?
- Comment assurer une visibilité en temps réel sur la disponibilité des ressources de coworking ?
- Comment automatiser le processus d'inscription et de paiement des formations, y compris pour des cours individuels ?
- Quels outils et technologies utiliser pour garantir la fiabilité, la sécurité et la performance de l'application ?

En répondant à ces questions, ce projet a pour objectif de proposer une solution web complète, moderne et évolutive, facilitant la gestion quotidienne des activités de coworking et de formation de Logicom.

Le tableau 1.2 résume les problèmes identifiés et les solutions proposées en réponse.

Problèmes identifiés	Solutions proposées
Gestion manuelle ou non centralisée des inscriptions aux formations, entraînant des erreurs et des doublons.	Mise en place d'un système d'inscription en ligne automatisé et centralisé.
Manque de visibilité en temps réel sur les réservations des espaces de coworking, causant des conflits.	Développement d'un système de réservation en temps réel avec affichage des disponibilités.
Difficulté à suivre les utilisateurs, formations et réservations de manière	Centralisation de toutes les données dans une base de données unique et sécurisée.

Problèmes identifiés	Solutions proposées
globale.	
Tâches administratives répétitives et chronophages pour les administrateurs.	Création d'un tableau de bord administrateur pour simplifier la gestion et automatiser certaines tâches.
Mauvaise expérience utilisateur due à un manque de structure et d'organisation.	Conception d'une interface intuitive, ergonomique et facile à utiliser.

Table 1.2 – Problèmes identifiés et solutions proposées

1.4.3 Étude de l'existant

Avant d'entamer la conception de notre solution, il est primordial de commencer par l'analyse des plateformes existantes sur le marché du coworking et de la formation en ligne, suivie d'une évaluation critique visant à identifier leurs limites.

1.4.3.1 Solutions existantes

a) Solutions de gestion d'espaces de coworking

Cobot : Cobot est une plateforme de gestion d'espaces de coworking permettant la réservation de bureaux et de salles de réunion, la gestion des membres via des forfaits personnalisés, ainsi que l'intégration de systèmes de facturation, de contrôle d'accès et de paiement. Elle propose également une application mobile dédiée à la réservation rapide des espaces.

Avantages : interface riche en intégrations (facturation, accès, calendrier), gestion fine des tarifs et de la capacité des ressources, application mobile native.

Limites : solution payante orientée grands espaces de coworking professionnels, complexité de configuration élevée pour une structure de taille moyenne combinant coworking et formation, absence de module de formation intégré.

Archie : Archie est une plateforme de gestion des espaces de travail flexibles, classée parmi les solutions les mieux notées du secteur. Elle permet la réservation de bureaux et de salles via application mobile, plateforme web ou kiosque en libre-service, la gestion des adhésions, de la facturation automatisée et des prospects.

Avantages : automatisation poussée des réservations, paiements et notifications ; rapports en temps réel sur l'occupation des espaces ; gestion des prospects intégrée.

Limites : solution généraliste non spécialisée dans la combinaison coworking/formation, coût élevé pour une petite structure, nécessite une intégration tierce pour gérer un volet pédagogique.

b) Solutions de gestion de formation en ligne (LMS)

Teachizy : Teachizy est une plateforme française de type LMS (Learning Management System) permettant de créer, diffuser et vendre des formations en ligne. Elle propose des pages de vente personnalisables, un paiement en plusieurs mensualités, des tableaux de bord de suivi par apprenant et des automatisations pour les inscriptions.

Avantages : gestion fine des inscriptions et des paiements, personnalisation des contenus pédagogiques (vidéos, quiz, fichiers), automatisation des communications.

Limites : plateforme entièrement dédiée à la formation, sans aucune fonctionnalité de gestion d'espace physique ou de réservation de salles ; nécessiterait l'utilisation d'un second outil pour le volet coworking.

LearnyBox : LearnyBox est un LMS français permettant de créer, vendre et animer des formations en ligne, avec une gestion intégrée de la facturation, des contenus pédagogiques (SCORM, quiz, vidéos) et de la conformité réglementaire (Qualiopi).

Avantages : solution complète pour la vente et le suivi des formations, personnalisation de la marque, gestion automatisée des paiements.

Limites : interface jugée peu intuitive pour les nouveaux utilisateurs, aucune gestion d'espaces physiques ou de réservation, coût mensuel élevé pour une structure combinant plusieurs services.

1.4.3.2 Critique de l'existant

L'analyse des solutions existantes sur le marché du coworking et de la formation en ligne a permis d'identifier plusieurs limitations récurrentes :

- **Fragmentation des outils** : aucune des solutions étudiées ne propose une gestion unifiée du coworking et de la formation au sein d'une seule et même plateforme. Les structures souhaitant offrir les deux services doivent combiner plusieurs outils, ce qui complexifie l'administration et alourdit les coûts.
- **Manque de centralisation des données** : les informations relatives aux utilisateurs, aux réservations et aux inscriptions sont dispersées entre différents systèmes, rendant le suivi global difficile pour les administrateurs.
- **Rigidité des modules de réservation** : les systèmes de réservation de coworking existants ne permettent pas toujours une granularité fine (gestion hiérarchique salle / emplacement / table) adaptée à des espaces modulaires.
- **Absence de panier mixte** : aucune solution de formation analysée ne permet de combiner, dans un même processus d'achat, l'inscription à une formation complète et à des cours individuels.

- **Coût et complexité** : les solutions spécialisées (Archie, Cobot, LearnyBox) s'adressent principalement à de grandes structures, avec des tarifs et une complexité de configuration peu adaptés à un besoin interne comme celui de Logicom.

1.4.4 Solution proposée

Afin de répondre aux limitations identifiées dans les solutions existantes, le projet TZ Prime Solutions propose les améliorations suivantes :

- **Plateforme unifiée** : une seule application web regroupant à la fois la gestion du coworking et celle des formations, évitant ainsi la fragmentation des outils.
- **Modèle hiérarchique de réservation** : une structuration des espaces de coworking en trois niveaux (Salle → Emplacement → Table), permettant une réservation précise d'une salle complète, d'un emplacement ou d'une simple table, avec un cycle de validation par l'administrateur (en attente, acceptée, refusée).
- **Catalogue de formations flexible** : un module Formation organisé par catégories (Tech, Créatif, Business, IA & Data, Entrepreneuriat & Startup), permettant l'inscription à une formation complète ou à des cours individuels.
- **Panier mixte et inscription centralisée** : un système de panier permettant de combiner formations et cours individuels dans un même processus de validation, avec une gestion différenciée des cas de succès partiel ou d'inscription déjà existante.
- **Tableau de bord administrateur** : une interface dédiée permettant aux administrateurs de gérer les salles, les emplacements, les tables, les formations et de valider les réservations et inscriptions, réduisant ainsi la charge administrative.
- **Base de données centralisée et sécurisée** : toutes les données relatives aux utilisateurs, réservations et inscriptions sont stockées dans une base unique, garantissant cohérence et traçabilité.

1.5 Présentation de la méthode agile « Scrum »

Dans le cadre de notre projet de développement de la plateforme TZ Prime Solutions, nous avons adopté la méthode agile Scrum afin de garantir une gestion souple, collaborative et itérative du travail. Cette méthode nous a permis de diviser le développement en sprints courts, chacun orienté vers un ensemble de fonctionnalités spécifiques telles que la gestion des réservations de coworking, la gestion des formations, ou encore l'interface d'administration. À la fin de chaque sprint, une version fonctionnelle et testable de l'application a été livrée, ce qui a favorisé les retours rapides et les ajustements nécessaires pour mieux répondre aux besoins réels de Logicom.

1.5.1 Les artefacts Scrum

- **Le backlog du produit** : c'est la liste complète des fonctionnalités, besoins et améliorations à intégrer dans la plateforme (module Coworking, module Formation, interface d'administration, système de paiement).
- **Le backlog du sprint** : c'est la sélection d'éléments du backlog produit que l'équipe s'engage à réaliser durant un sprint (par exemple, la création complète du module de réservation des espaces de coworking).
- **L'incrément du produit** : c'est la version fonctionnelle et potentiellement livrable de l'application à la fin d'un sprint (par exemple, après un sprint, une interface de réservation de salles opérationnelle est disponible pour test).

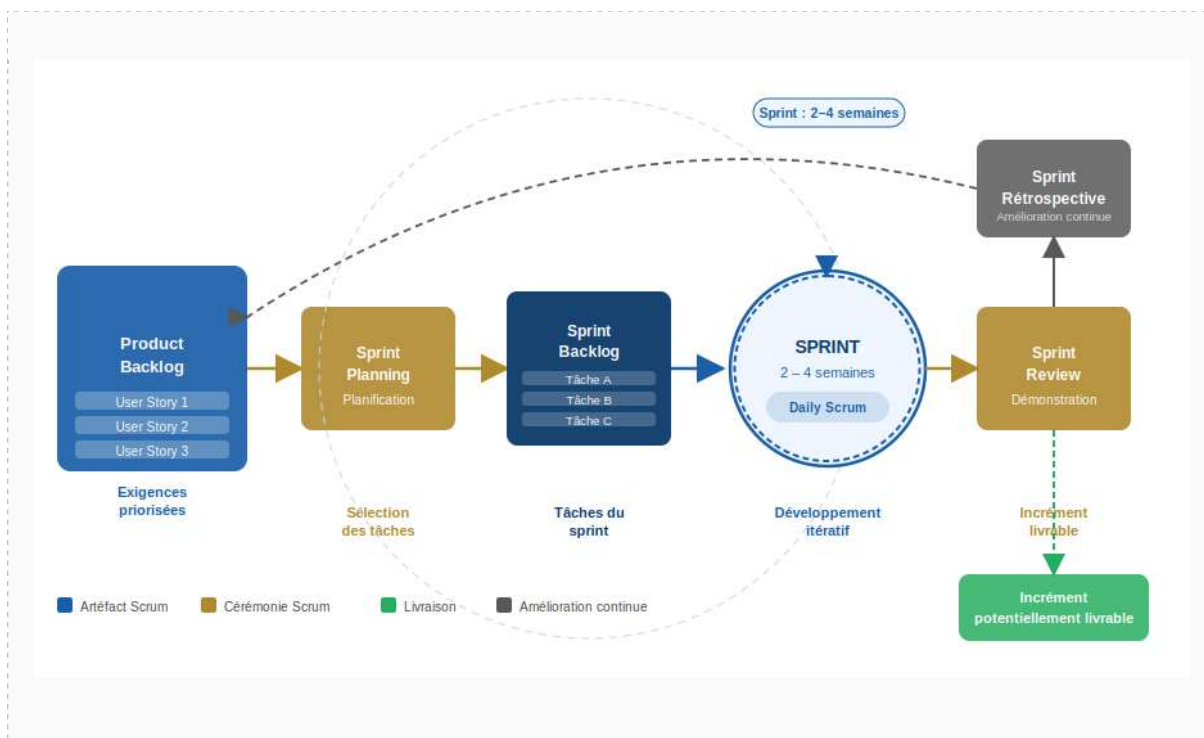


Figure 1.2. Cycle de vie Scrum

1.5.2 Les rôles de la méthode Scrum

La méthodologie Scrum met l'accent sur la notion de partage et favorise une meilleure cohérence d'équipe. Elle nécessite l'intervention de trois rôles principaux :

- **Product Owner** : responsable de la définition et de la priorisation des fonctionnalités à développer, en représentant les intérêts de Logicom et des futurs utilisateurs de la plateforme.
- **Scrum Master** : facilitateur essentiel de la méthodologie, chargé de soutenir l'équipe de développement en éliminant les obstacles et en facilitant les réunions de suivi, les revues de sprint et les rétrospectives.
- **Équipe de développement** : groupe de professionnels (ici, les étudiants en charge du projet) responsables de la conception, du développement et des tests des différents modules de la plateforme.

1.6 Conclusion

Dans ce premier chapitre, nous avons présenté la société d'accueil Logicom ainsi que ses secteurs d'activités. Nous avons ensuite exposé le contexte et la problématique ayant motivé le développement de la plateforme TZ Prime Solutions, à travers une étude critique des solutions existantes sur le marché du coworking et de la formation en ligne. Cette analyse a mis en évidence l'absence de solution unifiée combinant ces deux services, justifiant ainsi la pertinence de notre projet. Enfin, nous avons présenté la méthodologie agile Scrum adoptée pour le pilotage du projet. Le chapitre suivant sera consacré à l'analyse et à la spécification des besoins fonctionnels et non-fonctionnels de la plateforme.

Chapitre 2

Analyse et spécification des besoins

2.1 Introduction

Après avoir présenté, dans le chapitre précédent, l'organisme d'accueil, le contexte ainsi que la problématique de notre projet, nous nous intéressons dans ce chapitre à l'analyse et à la spécification des besoins de la plateforme TZ Prime Solutions. Nous commencerons par une synthèse de l'analyse de l'existant qui a permis de capturer les besoins réels des futurs utilisateurs. Nous identifierons ensuite les différents acteurs intervenant dans le système, avant de détailler les spécifications fonctionnelles propres à chacun des deux modules de la plateforme : le module Co-Working et le module Formation. Nous terminerons ce chapitre par l'énumération des besoins non-fonctionnels devant être respectés par la solution.

2.2 Analyse de l'existant et capture des besoins

L'étude des solutions existantes, présentée au chapitre précédent, a mis en évidence l'absence d'une plateforme unifiée combinant à la fois la gestion d'un espace de coworking et celle de programmes de formation. Cette analyse, combinée à des échanges avec l'encadrant professionnel et à l'observation des pratiques internes de Logicom, a permis de dégager un ensemble de besoins concrets que la plateforme TZ Prime Solutions doit satisfaire.

Du côté du **coworking**, le besoin central est la mise en place d'un système de réservation hiérarchisé, capable de représenter fidèlement l'organisation physique des locaux : un site se compose de salles, chaque salle comporte un ou plusieurs emplacements, et chaque emplacement regroupe un ensemble de tables. L'utilisateur doit pouvoir réserver soit une salle entière (avec sélection optionnelle des tables qu'il souhaite occuper), soit une table individuelle, tandis que l'administrateur doit conserver un contrôle total sur la validation de ces réservations afin d'éviter tout conflit d'occupation.

Du côté de la **formation**, le besoin principal est de proposer un catalogue de formations organisées par catégories thématiques (Tech, Créatif, Business, IA & Data, Entrepreneuriat & Startup), avec la possibilité pour l'utilisateur de s'inscrire soit à une formation complète, soit à des cours individuels qui la composent. Le processus d'achat doit pouvoir combiner ces deux types d'éléments dans un même panier, avant validation par paiement.

Enfin, un besoin transversal aux deux modules est la nécessité d'un tableau de bord administrateur permettant de centraliser la gestion des ressources de coworking, du catalogue de formations, ainsi que la validation des réservations et des inscriptions, réduisant ainsi la charge de travail manuelle des administrateurs.

2.3 Identification des acteurs

La plateforme TZ Prime Solutions met en relation plusieurs types d'utilisateurs, chacun disposant de droits d'accès et de fonctionnalités spécifiques. Le tableau 2.1 présente les différents acteurs identifiés ainsi que leur rôle au sein du système.

Acteur	Description
Visiteur	Utilisateur non authentifié pouvant consulter le catalogue des formations proposées, mais devant créer un compte ou s'authentifier pour effectuer une réservation ou une inscription.
Membre	Utilisateur authentifié pouvant consulter et réserver les ressources de coworking (salles, emplacements, tables), consulter le catalogue de formations, ajouter des formations ou des cours à son panier et finaliser ses inscriptions, et suivre l'état de ses réservations et inscriptions.
Administrateur	Utilisateur disposant de droits étendus : gestion complète (ajout, modification, suppression) des salles, emplacements et tables, gestion du catalogue de formations, et validation ou refus des demandes de réservation effectuées par les membres.
Superadministrateur	Utilisateur disposant des mêmes droits que l'administrateur, avec en plus la gestion des comptes administrateurs et la supervision globale de la plateforme.

Table 2.1 – Identification des acteurs

Sur le plan technique, l'application distingue ces rôles à travers un attribut « role » associé à chaque utilisateur (admin, superadmin, ou utilisateur standard), ce qui conditionne dynamiquement l'affichage des interfaces et des actions disponibles. Par exemple, un administrateur ne voit pas l'onglet « Mes Réservations » dans le module Co-Working, puisque la validation des réservations relève de son propre tableau de bord, et non d'un suivi personnel.

2.4 Spécifications fonctionnelles

Cette section détaille les besoins fonctionnels propres à chacun des deux modules constituant le cœur de la plateforme TZ Prime Solutions.

2.4.1 Module Co-Working

Le module Co-Working a pour objectif de permettre la gestion et la réservation des espaces de travail collaboratif. Il repose sur une structuration hiérarchique des ressources à trois niveaux : une **Salle** regroupe un ou plusieurs **Emplacements**, et chaque emplacement contient un ensemble de **Tables**.

a) Besoins fonctionnels côté Administrateur

- Consulter la liste des salles, emplacements et tables disponibles.
- Ajouter une nouvelle salle, en spécifiant son nom, sa capacité d'accueil et sa disponibilité.
- Ajouter un nouvel emplacement, en l'associant à une salle existante et en spécifiant son nombre de places.
- Ajouter une nouvelle table, en l'associant à un emplacement existant et en définissant son statut (Libre, Occupée, Réservée).
- Modifier ou supprimer une salle, un emplacement ou une table existants.
- Valider ou refuser les demandes de réservation soumises par les membres, depuis le tableau de bord administrateur.

b) Besoins fonctionnels côté Membre

- Consulter la liste des salles disponibles, avec leur capacité et leur statut de disponibilité.
- Consulter la liste des emplacements, filtrée par salle.
- Consulter la liste des tables, avec leur statut en temps réel (Libre, Occupée, Réservée).
- Réserver une salle complète pour une date et une plage horaire données, avec la possibilité de sélectionner les tables spécifiques de cette salle à occuper.
- Réserver une table individuelle pour une date et une plage horaire données.
- Consulter la liste de ses propres réservations, avec leur statut (en attente, acceptée, refusée).
- Annuler une réservation en attente ou acceptée.

c) Cycle de vie d'une réservation

Toute demande de réservation soumise par un membre est initialement enregistrée avec le statut « **en attente** ». L'administrateur examine ensuite la demande et peut la faire passer à l'état « **acceptée** » ou « **refusée** ». Ce mécanisme de validation manuelle permet d'éviter les doubles réservations et de garantir une occupation cohérente des espaces physiques, tout en laissant à l'administrateur la possibilité d'arbitrer en cas de conflit. La figure 2.1 illustre ce cycle de vie.

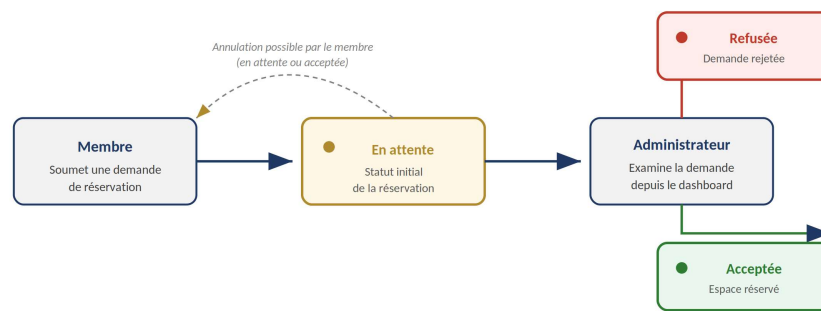


Figure 2.1. Cycle de vie d'une réservation de coworking

2.4.2 Module Formation

Le module Formation a pour objectif de permettre la consultation, l'inscription et le suivi des formations proposées par Logicom. Chaque formation appartient à une catégorie thématique et peut être composée de plusieurs cours individuels pouvant être suivis indépendamment.

a) Besoins fonctionnels côté Administrateur

- Consulter le catalogue complet des formations.
- Ajouter, modifier ou supprimer une formation (titre, description, prix, catégorie, cours associés).
- Consulter les inscriptions des membres aux formations et aux cours individuels.
- Valider les inscriptions en attente, le cas échéant, depuis le tableau de bord administrateur.

b) Besoins fonctionnels côté Membre

- Consulter le catalogue des formations, avec filtrage par catégorie (Tech, Créatif, Business, IA & Data, Entrepreneuriat & Startup).
- Consulter le détail d'une formation, incluant la liste des cours individuels qui la composent.
- Ajouter une formation complète ou un cours individuel à un panier d'achat.
- Retirer un élément du panier, ou vider entièrement le panier.
- Finaliser ses inscriptions en validant le contenu du panier (paiement et confirmation).
- Consulter la liste de ses formations et cours auxquels il est inscrit.

c) Gestion du panier et du processus d'inscription

Le panier permet de combiner, dans une même démarche d'achat, des formations complètes et des cours individuels. Lors de la validation, le système traite séparément les inscriptions aux formations et celles aux cours individuels, et est conçu pour gérer de façon résiliente les cas où certains éléments du panier auraient déjà fait l'objet d'une inscription préalable : dans ce cas, seuls les éléments non encore inscrits sont traités, et

l'utilisateur reçoit un message différencié selon que l'opération a totalement, partiellement, ou pas du tout réussi. Cette robustesse évite qu'une erreur isolée (par exemple une inscription déjà existante) ne bloque la validation des autres éléments du panier.

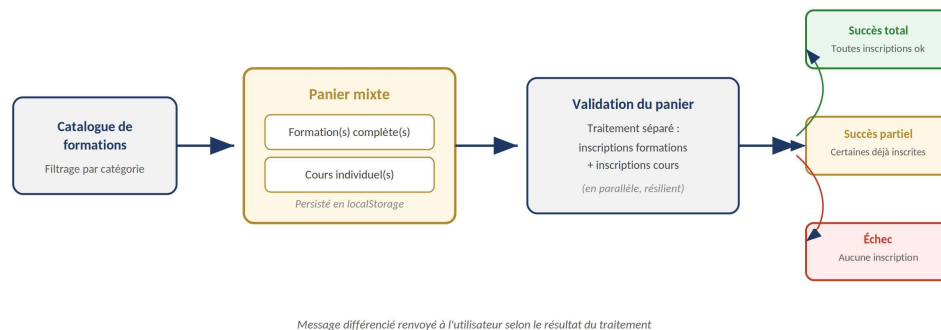


Figure 2.2. Processus d'inscription à une formation ou à un cours

2.5 Besoins non-fonctionnels

Au-delà des fonctionnalités métier décrites précédemment, la plateforme TZ Prime Solutions doit satisfaire un ensemble d'exigences de qualité, résumées dans le tableau 2.2.

Besoin fonctionnel	non-	Description
Sécurité		L'accès aux fonctionnalités est protégé par un mécanisme d'authentification basé sur un jeton (token), transmis dans l'en-tête des requêtes. Les actions sensibles (création, modification, suppression) sont restreintes selon le rôle de l'utilisateur.
Performance		Les données nécessaires à l'affichage d'une page (par exemple, les salles, emplacements, tables et réservations du module Co-Working) sont récupérées en parallèle afin de réduire le temps de chargement perçu par l'utilisateur.
Ergonomie et utilisabilité	et	L'interface doit être intuitive, organisée par onglets thématiques, avec des retours visuels clairs (messages de succès, d'erreur, badges de statut colorés) pour guider l'utilisateur dans ses actions.
Disponibilité en temps réel	en	Le statut des ressources de coworking (Libre, Occupée, Réservée) et des inscriptions doit refléter l'état réel du système à tout moment, afin d'éviter les conflits de réservation.

Besoin fonctionnel	non-	Description
Portabilité		L'application, développée sous forme de Single Page Application (SPA) avec React, doit être accessible depuis différents navigateurs web modernes, sans installation préalable.
Maintenabilité		Le code est organisé en composants réutilisables et modulaires (un composant dédié par module fonctionnel), afin de faciliter les évolutions futures de la plateforme.
Scalabilité		L'architecture (frontend React / backend Node.js / API REST) doit permettre l'ajout futur de nouveaux modules ou de nouvelles ressources sans remise en cause de la structure existante.
Cohérence des données	des	Toute action de réservation ou d'inscription doit être immédiatement répercutée dans la base de données, afin de garantir que tous les utilisateurs disposent d'une vision cohérente de l'état du système.

Table 2.2 – Besoins non-fonctionnels

2.6 Conclusion

Ce chapitre a permis de poser les bases fonctionnelles de la plateforme TZ Prime Solutions. Nous avons d'abord synthétisé les besoins issus de l'analyse de l'existant, puis identifié les différents acteurs amenés à interagir avec le système. Nous avons ensuite détaillé les spécifications fonctionnelles des deux modules principaux, Co-Working et Formation, en mettant l'accent sur leurs mécanismes propres : la hiérarchie Salle/Emplacement/Table et le cycle de validation des réservations d'une part, le catalogue par catégories et le panier mixte d'inscription d'autre part. Enfin, nous avons énuméré les besoins non-fonctionnels garantissant la qualité globale de la solution. Le chapitre suivant sera consacré à la conception de la solution, à travers la définition de l'architecture technique et du schéma de la base de données.

Chapitre 3

Conception de la solution

Ce chapitre présente l'ensemble des décisions de conception qui sous-tendent le Système de Formations. Nous y détaillons l'architecture technique adoptée, la modélisation de la base de données issue du diagramme de classes UML, ainsi que la logique métier orchestrant les interactions entre les différents modules fonctionnels.

3.1 Architecture technique

L'architecture du système repose sur un modèle multicouches (n-tiers) qui sépare clairement les responsabilités entre la présentation, la logique métier et la persistance des données. Ce choix architectural garantit la maintenabilité, l'évolutivité et la testabilité de l'application.

3.1.1 *Vue d'ensemble des couches*

Le système est structuré autour de trois couches principales :

- **Couche Présentation (Front-end)** : interface utilisateur accessible via navigateur web, permettant aux Utilisateurs, Admins et SuperAdmins d'interagir avec le système selon leurs droits respectifs.
- **Couche Métier (Back-end)** : API REST assurant le traitement des règles métier, la gestion des sessions d'authentification (token JWT), le contrôle d'accès basé sur les rôles (RBAC) et l'orchestration des workflows.
- **Couche Données (Persistance)** : base de données relationnelle stockant l'ensemble des entités du domaine — Utilisateurs, Formations, Inscriptions, Réservations, Salles, Espaces, Centres et Packs.

3.1.2 *Acteurs et rôles du système*

Le diagramme des cas d'utilisation ci-dessous illustre les trois niveaux d'acteurs identifiés et leurs interactions avec le système :

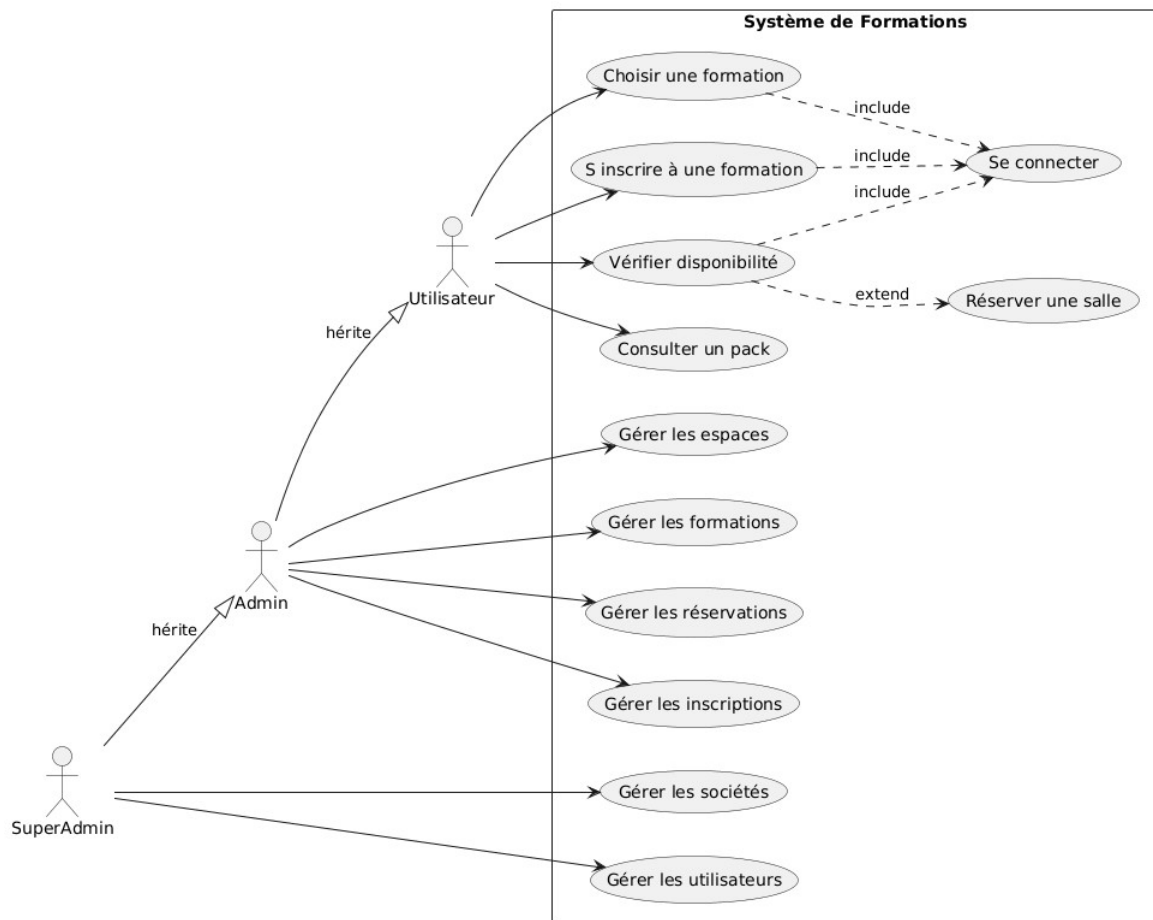


Figure 3.1. Diagramme des cas d'utilisation du Système de Formations

Utilisateur	Acteur de base pouvant choisir et s'inscrire à une formation, vérifier la disponibilité d'une salle et consulter un pack. Toute action nécessite une connexion préalable (include → Se connecter).
Admin	Hérite des droits Utilisateur et dispose en plus des fonctions de gestion : espaces, formations, réservations et inscriptions.
SuperAdmin	Hérite des droits Admin et dispose des droits exclusifs de gestion des sociétés et des utilisateurs.

3.1.3 Principes d'authentification et de sécurité

Le mécanisme d'authentification s'appuie sur la classe Authentification qui génère un token JWT à durée limitée. Chaque requête vers les ressources protégées doit présenter un token valide ; la méthode invalider() permet de révoquer une session. Le contrôle d'accès est appliqué côté serveur en fonction du rôle de l'utilisateur

3.2 Conception de la base de données et schéma relationnel

Le schéma relationnel est directement dérivé du diagramme de classes UML présenté ci-dessous. Chaque classe persistante correspond à une table, les associations sont traduites en clés étrangères et les multiplicités définissent les contraintes d'intégrité référentielle.

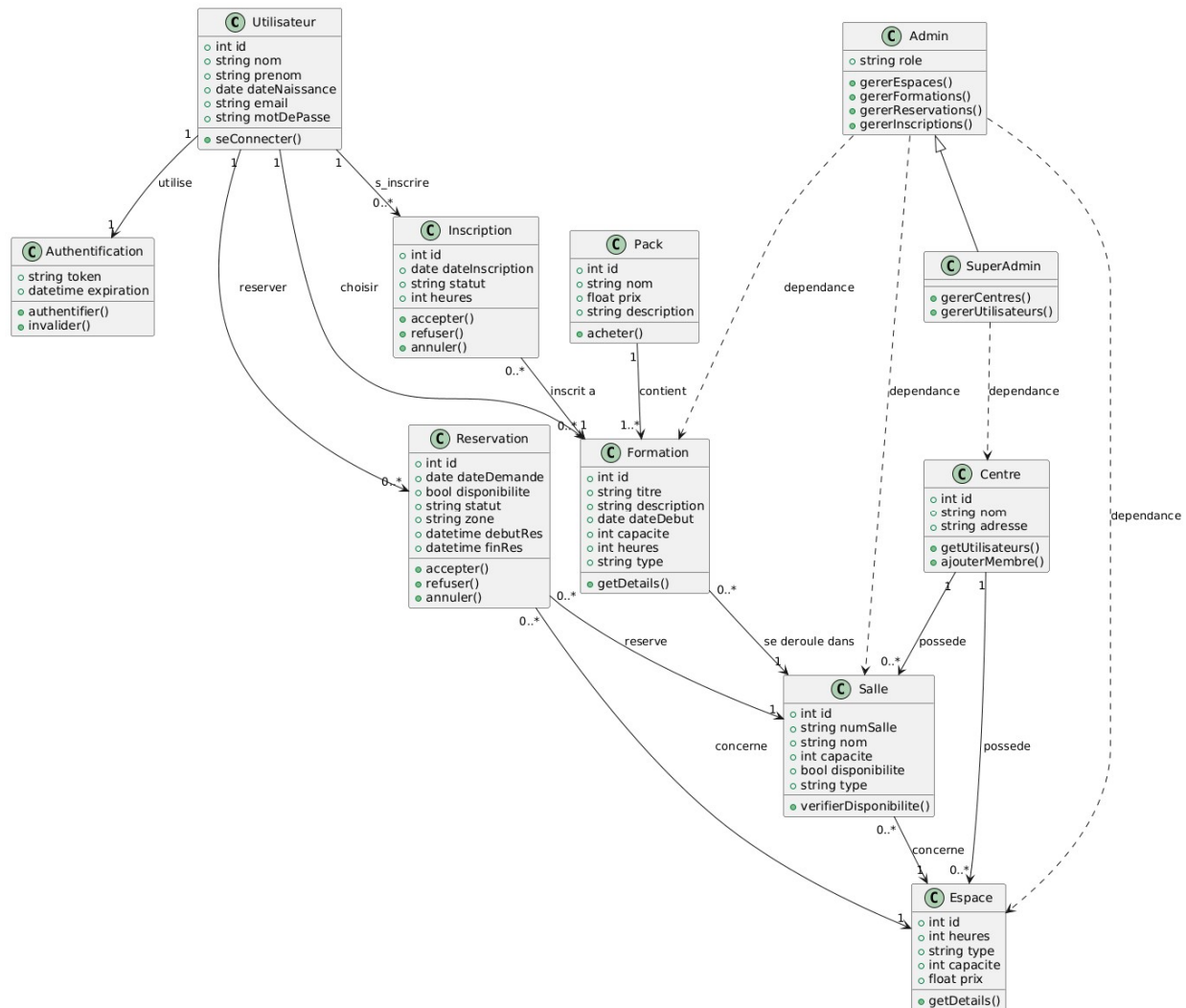


Figure 3.2. Diagramme de classes UML du Système de Formations

3.2.1 Tables principales

Le schéma comprend neuf entités principales dont les définitions sont détaillées ci-après.

Table : utilisateur

Champ	Type	Contrainte	Description
id	INT	PK / AUTO_INCREMENT	Identifiant unique
nom	VARCHAR(100)	NOT NULL	Nom de famille
prenom	VARCHAR(100)	NOT NULL	Prénom
dateNaissance	DATE		Date de naissance
email	VARCHAR(150)	NOT NULL / UNIQUE	Adresse e-mail
motDePasse	VARCHAR(255)	NOT NULL	Mot de passe haché

Table : authentication

Champ	Type	Contrainte	Description
id	INT	PK / AUTO_INCREMENT	Identifiant de session
token	VARCHAR(512)	NOT NULL	Token JWT
expiration	DATETIME	NOT NULL	Date/heure d'expiration
utilisateur_id	INT	FK → utilisateur.id	Propriétaire du token

Table : formation

Champ	Type	Contrainte	Description
id	INT	PK / AUTO_INCREMENT	Identifiant unique
titre	VARCHAR(200)	NOT NULL	Intitulé de la formation
description	TEXT		Description détaillée
dateDebut	DATE	NOT NULL	Date de début
capacite	INT	NOT NULL	Nombre max. de participants
heures	INT	NOT NULL	Durée en heures
type	VARCHAR(50)		Présentiel /

Champ	Type	Contrainte	Description
			distanciel / mixte
salle_id	INT	FK → salle.id	Salle d'accueil

Table : inscription

Champ	Type	Contrainte	Description
id	INT	PK / AUTO_INCREMENT	Identifiant unique
dateInscription	DATE	NOT NULL	Date de la demande
statut	VARCHAR(30)	NOT NULL	En attente / Acceptée / Refusée
heures	INT		Heures validées
utilisateur_id	INT	FK → utilisateur.id	Candidat
formation_id	INT	FK → formation.id	Formation ciblée

Table : reservation

Champ	Type	Contrainte	Description
id	INT	PK / AUTO_INCREMENT	Identifiant unique
dateDemande	DATE	NOT NULL	Date de la demande
disponibilite	BOOLEAN	NOT NULL	Disponibilité confirmée
statut	VARCHAR(30)	NOT NULL	En attente / Acceptée / Refusée
zone	VARCHAR(100)		Zone géographique
debutRes	DATETIME	NOT NULL	Début de réservation
finRes	DATETIME	NOT NULL	Fin de réservation
utilisateur_id	INT	FK → utilisateur.id	Demandeur
salle_id	INT	FK → salle.id	Salle réservée

Champ	Type	Contrainte	Description
formation_id	INT	FK → formation.id	Formation concernée

Table : salle

Champ	Type	Contrainte	Description
id	INT	PK / AUTO_INCREMENT	Identifiant unique
numSalle	VARCHAR(20)	NOT NULL / UNIQUE	Numéro de salle
nom	VARCHAR(100)	NOT NULL	Nom de la salle
capacite	INT	NOT NULL	Capacité d'accueil
disponibilite	BOOLEAN	NOT NULL	Disponible ou non
type	VARCHAR(50)		Amphi / TD / TP...
centre_id	INT	FK → centre.id	Centre propriétaire

Table : espace

Champ	Type	Contrainte	Description
id	INT	PK / AUTO_INCREMENT	Identifiant unique
heures	INT		Créneaux disponibles
type	VARCHAR(50)		Type d'espace
capacite	INT		Capacité
prix	FLOAT		Tarif horaire
salle_id	INT	FK → salle.id	Salle parente
centre_id	INT	FK → centre.id	Centre propriétaire

Table : pack

Champ	Type	Contrainte	Description
id	INT	PK / AUTO_INCREMENT	Identifiant unique
nom	VARCHAR(100)	NOT NULL	Nom du pack
prix	FLOAT	NOT NULL	Prix TTC
description	TEXT		Contenu du pack

Table : centre

Champ	Type	Contrainte	Description
id	INT	PK / AUTO_INCREMENT	Identifiant unique
nom	VARCHAR(150)	NOT NULL	Nom du centre
adresse	TEXT	NOT NULL	Adresse complète

3.3 Logique métier et relations inter-modules

La logique métier orchestre les interactions entre les modules du système. Cette section décrit les flux principaux à travers le diagramme de séquence ainsi que les règles de gestion associées.

3.3.1 Diagramme de séquence – Inscription à une formation

Le diagramme de séquence ci-dessous modélise le scénario nominal d'inscription d'un utilisateur à une formation, depuis l'authentification jusqu'à la confirmation ou le refus par l'administrateur.

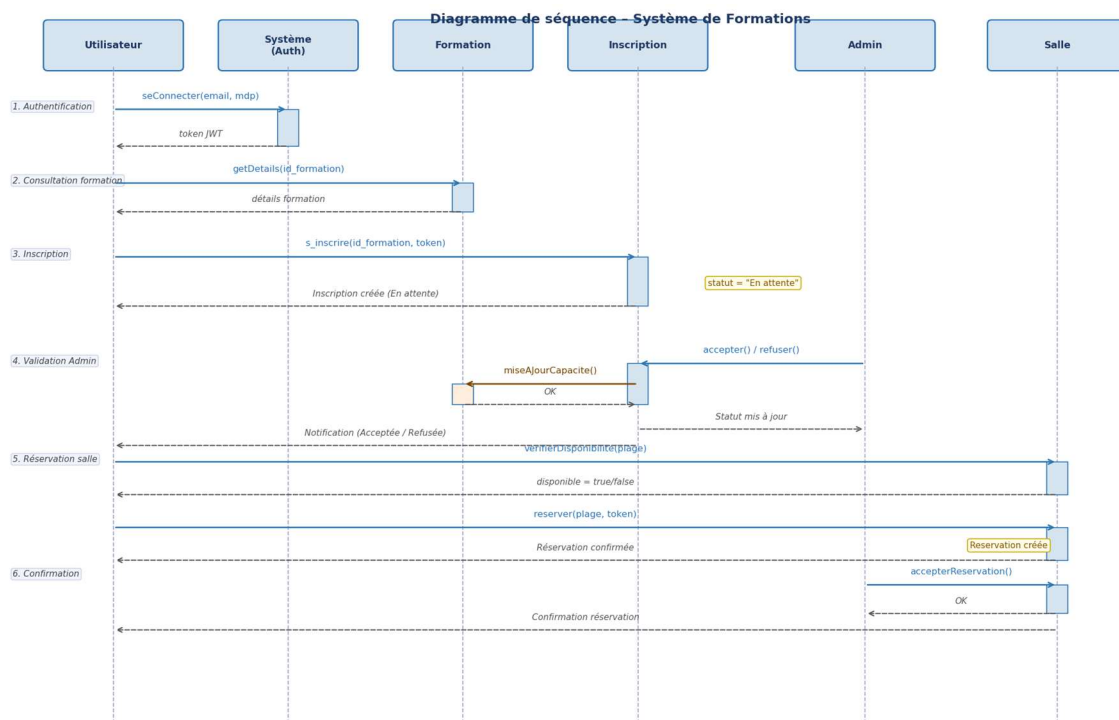


Figure 3.3. Diagramme de séquence : Inscription à une formation

3.3.2 Flux d'inscription à une formation

Le processus d'inscription suit les étapes suivantes :

- L'utilisateur s'authentifie via `seConnecter()` ; le système génère un token JWT (Authentification).
- L'utilisateur consulte les formations disponibles et en choisit une (choisir).
- Une inscription est créée avec le statut « En attente » (`s_inscrire` → Inscription).
- L'Admin examine la demande et invoque `accepter()` ou `refuser()` sur l'objet Inscription.

- En cas d'acceptation, la capacité de la Formation est décrétementée et l'utilisateur est notifié.

3.3.3 Flux de réservation de salle

La réservation d'une salle est un processus étendu (extend) du cas « Vérifier disponibilité » :

- L'utilisateur appelle `verifierDisponibilite()` sur la Salle cible pour une plage horaire donnée.
- Si la salle est disponible (`disponibilite = true`), une Reservation est créée à l'état « En attente ».
- L'Admin valide ou rejette la réservation via `accepter()` / `refuser()` / `annuler()`.
- La disponibilité de la salle est mise à jour en conséquence.

3.3.4 Relations inter-modules et dépendances

Les dépendances entre modules reflètent la hiérarchie organisationnelle du système :

- Le SuperAdmin supervise les Centres ; chaque Centre possède des Salles et des Espaces.
- L'Admin gère les Formations, Inscriptions, Réservations et Espaces au sein de son périmètre.
- Une Formation se déroule dans une Salle et peut être contenue dans un Pack.
- Une Réservation concerne à la fois une Salle et une Formation, et est liée à un Espace.
- L'Authentification est utilisée par l'Utilisateur pour sécuriser toutes ses interactions.

3.3.5 Règles de gestion principales

Règle	Description
RG-01	Un utilisateur ne peut s'inscrire à une formation que s'il est authentifié (token valide non expiré).
RG-02	Une formation ne peut excéder sa capacité maximale ; toute inscription supplémentaire est refusée automatiquement.
RG-03	Une salle ne peut être réservée que si <code>disponibilite = true</code> sur la plage horaire demandée.
RG-04	Seul un Admin ou SuperAdmin peut accepter, refuser ou annuler une inscription ou une réservation.
RG-05	Un Pack peut contenir plusieurs formations (1..*) ; une formation peut appartenir à un seul pack (0..1).
RG-06	Un Centre possède au moins une Salle ; une Salle appartient à un seul Centre.
RG-07	Le SuperAdmin est le seul à pouvoir créer ou supprimer des centres et des utilisateurs.

Conclusion

Ce chapitre a permis de formaliser l'ensemble des décisions de conception du Système de Formations. L'architecture multicouches adoptée assure une séparation nette des responsabilités. Le schéma relationnel, fidèle au diagramme de classes UML, garantit l'intégrité des données et la cohérence des associations entre entités. Enfin, la logique métier décrite à travers les diagrammes de séquence et les règles de gestion constitue le socle fonctionnel sur lequel reposera l'implémentation présentée au chapitre suivant.

Chapitre 4

Réalisation technique

4.1. Environnement de developpement et stack technologique

Le projet TZ Prime Solutions repose sur une architecture moderne de type Single Page Application (SPA) cote client, couplee a une API REST cote serveur. Ce choix architectural garantit une separation claire des responsabilites, une fluidite de navigation et une maintenabilite optimale.

4.1.1. Architecture generale

Architecture retenue : Client SPA (React) <-> API REST (Node.js/Express) <-> Base de donnees (MySQL).
Communication via JWT Bearer Token pour toutes les requetes authentifiees.

L'application est structuree en trois couches distinctes :

- Couche presentation : React.js avec React Router DOM v6 pour la navigation cote client
- Couche logique metier : Node.js + Express.js exposant des endpoints REST securises
- Couche donnees : Base de donnees relationnelle MySQL avec un ORM pour la gestion des entites

4.1.2. Stack Frontend

Framework UI React.js 18 — Functional Components + Hooks	Routage React Router DOM v6 — Navigate, useNavigate, useParams
Gestion d'etat useState, useEffect, useCallback — etat local React	Styling CSS Modules personnalises + Inline styles dynamiques
Communication API fetch() natif avec JWT Bearer Token dans les headers	Persistance locale localStorage — panier formations, historique achats

4.1.3. Stack Backend

Environnement Node.js + Express.js	Authentification JWT (JSON Web Tokens) — emission et verification
Base de donnees	Endpoint de base

MySQL — gestion relationnelle des entites	http://localhost:5000/api
Securite Authorization Bearer Token — middleware de verification	CORS & Headers Content-Type: application/json sur toutes les requetes

4.1.4. Outils de developpement

Editeur Visual Studio Code	Versionning Git / GitHub
Gestion de paquets npm	Debogage React DevTools + Console navigateur
Test API Postman / Thunder Client	Format de donnees JSON — echanges client/serveur standardises

4.2. Developpement du Frontend (Single Page Application)

L'interface utilisateur de TZ Prime Solutions est developpee en tant que Single Page Application, offrant une navigation fluide sans rechargement de page. Le frontend est articule autour de plusieurs composants principaux, chacun gerant un perimetre fonctionnel precis.

4.2.1. Architecture des composants React

Composant	Route	Role principal
ChooseServices	/choose-services	Page d'accueil post-connexion — selection du service
Formation	/formation	Catalogue des formations avec filtres et panier
FormationDetail	/formation/:id	Detail d'une formation et ses cours individuels
Coworking	/coworking	Gestion des salles, emplacements et tables
DashboardRouter	/admin ou /superadmin	Routeur de dashboard selon le role utilisateur
AdminDashboard	/admin & /superadmin	Panneau d'administration complet multi-onglets

4.2.2. Composant ChooseServices — Page d'accueil

Le composant ChooseServices constitue le point d'entree de l'application apres l'authentification. Il adapte dynamiquement son interface selon le role de l'utilisateur connecte.

- Detection du role : distinction entre user, admin et superadmin via user.role
- Navigation conditionnelle : affichage du bouton Dashboard Admin ou SuperAdmin uniquement pour les profils privileges
- Design en cartes : deux services presentes (Formation et Coworking) avec navigation directe via onClick
- Deconnexion : reset de l'etat utilisateur et redirection vers /login

4.2.3. Composant Formation — Catalogue et Panier

Le composant Formation gere l'affichage du catalogue de formations recuperees depuis l'API, ainsi qu'un systeme de panier persistant base sur localStorage.

Flux de donnees principal

- Recuperation des formations : GET /api/formations avec headers JWT
- Filtrage par categorie : Tech, Creatif, Business, IA & Data, Entrepreneuriat & Startup
- Assignment d'icônes dynamiques : analyse du titre et du tag de chaque formation
- Gestion du panier : localStorage avec cle cart_{userId} pour persistance inter-sessions

Systeme de panier

- Ajout/retrait d'une formation : toggleCart() avec verification de presence par id + _type
- Types d'articles supportes : formation (formation complete) et course (cours individuel)
- Persistance cross-composant : synchronisation via evenement storage entre Formation et FormationDetail
- Drawer lateral anime : affichage coulissant (slideInRight) avec recapitulatif et total TND

Processus de checkout

- Endpoint formations : POST /api/formations/inscrire-multiple avec tableau formationIds
- Endpoint cours individuels : POST /api/formations/{id}/cours/{courseId}/inscrire
- Gestion des conflits : detection des doublons (statut 409), feedback utilisateur differencie
- Resultat : vidage du panier, notification de succes, rechargement des donnees

4.2.4. Composant FormationDetail — Detail et Programmes

Le composant FormationDetail affiche le detail d'une formation specifique identifiee par son ID dans l'URL (useParams), ainsi que la liste de ses cours individuels.

- Recuperation : GET /api/formations/:id avec gestion du cas 404 (notFound state)
- Programmes statiques : objet coursesByCategory indexe par categorie DB (Tech, Creatif, Business, IA & Data...)
- Panier partage : meme cle localStorage cart_{userId} que Formation.jsx — coherence des donnees
- Niveaux de cours : Debutant (vert), Intermediaire (bleu), Avance (violet) avec color-coding
- Double ajout panier : formation complete OU cours individuel unitairement

Point technique notable : la synchronisation du panier entre Formation.jsx et FormationDetail.jsx s'appuie sur l'evenement natif window 'storage', permettant une mise a jour reactive sans context global ni Redux.

4.2.5. Composant Coworking — Espaces de travail

Le composant Coworking gere l'ensemble des operations relatives aux espaces de coworking : salles, emplacements, tables et reservations utilisateur.

Entites gerees

- Salles : entites principales avec capacite et statut disponible/indisponible
- Emplacements : zones au sein d'une salle, reliees par salle_id
- Tables : unites reservables, reliees a un emplacement, avec statut Libre / Occupee / Reservee
- Reservations : creees par l'utilisateur avec statut en_attente — validees par un admin

Gestion des droits

- Mode utilisateur : visualisation + bouton Reserver sur les entites disponibles, onglet Mes Reservations
- Mode admin : CRUD complet (Ajouter / Modifier / Supprimer), sans onglet Mes Reservations
- Detection du role : isAdmin = user.role === 'admin' || user.role === 'superadmin'

Flux de reservation utilisateur

```
POST /api/coworking/reservations

Body: { type, item_id, item_nom, date, heure_debut, heure_fin, statut: 'en_attente', table_ids
}
```

- Modale de reservation : selection de la date, heure debut/fin, tables disponibles (pour les salles)
- Selection multiple de tables : interface checkbox pour choisir les tables au sein d'une salle
- Annulation : DELETE /api/coworking/reservations/:id depuis l'onglet Mes Reservations

4.2.6. Routage et navigation — *DashboardRouter*

Le composant DashboardRouter implemente un pattern de routage base sur les roles, permettant d'afficher le dashboard approprié selon le profil connecté.

Role	Composant affiche	Comportement si non autorise
superadmin	SuperAdminDashboard	—
admin	AdminDashboard	—
user / non defini	Navigate to /choose-services	Redirection automatique
non connecte	Navigate to /login	Redirection authentication

4.3. Interface d'administration (Dashboard)

Le Dashboard d'administration constitue le coeur operationnel de la plateforme TZ Prime Solutions. Accessible aux profils admin et superadmin, il centralise la gestion de l'ensemble des donnees et actions administratives via une interface a onglets.

4.3.1. Architecture du Dashboard

Le composant AdminDashboard est un composant React monolithique de grande taille, gerant l'ensemble des entites via un systeme d'onglets. Il exploite les hooks `useState`, `useEffect` et `useCallback` pour la gestion d'etat et les effets de bord.

Donnees chargees au montage

- Utilisateurs : `GET /api/admin/users`
- Reservations coworking : `GET /api/coworking/reservations`
- Inscriptions formations : `GET /api/formations/inscriptions/all`
- Historique achats : lecture depuis `localStorage` (cle `tz_purchases_history`)

Optimisation : le chargement initial utilise `Promise.all()` pour paralleliser les 3 requetes API, reduisant le temps d'attente global vs. des appels sequentiels.

4.3.2. Systeme d'onglets — 6 sections fonctionnelles

Onglet	Contenu principal	Badge
Utilisateurs	Tableau CRUD des comptes utilisateurs	Nombre total
Reservations	Demandes en attente de validation coworking	En attente
Inscriptions	Demandes en attente de validation formation	En attente
Achats	Historique des commandes	Total commandes

	formations	
Historique	Decisions prises (acceptees + refusees)	Nombre entrees
Statistiques	KPIs, graphiques barres, donut statuts	—

4.3.3. Onglet Utilisateurs — *CRUD Complet*

L'onglet Utilisateurs offre une gestion complete des comptes avec recherche, filtres et actions directes.

Filtres disponibles

- Recherche textuelle : filtrage en temps reel sur name et email
- Filtre par role : Tous / Utilisateur / Admin / Superadmin
- Filtre par statut : Tous / Actif / Inactif / Suspendu

Actions disponibles selon le role

Action	Admin	SuperAdmin	Endpoint
Modifier utilisateur	Oui	Oui	PUT /api/admin/users/:id/statut
Suspendre/Activer	Oui	Oui	PUT /api/admin/users/:id/statut
Changer le role	Non	Oui	PUT /api/admin/users/:id/role
Supprimer definitivement	Non	Oui	DELETE /api/admin/users/:id

4.3.4. Onglets Reservations & Inscriptions

Ces deux onglets affichent exclusivement les demandes en statut en_attente, offrant des actions binaires d'acceptation ou de refus.

- Separation active/historique : les demandes traitees migrent automatiquement dans l'onglet Historique

- Accepter : PUT /api/coworking/reservations/:id/statut avec body { statut: 'acceptee' }
- Refuser : PUT /api/coworking/reservations/:id/statut avec body { statut: 'refusee' }
- Supprimer (superadmin) : DELETE /api/coworking/reservations/:id
- Interface visuelle : bordure gauche orange (#f59e0b) pour les elements en attente

4.3.5. Onglet Achats — Commandes Formations

L'onglet Achats presente l'historique des commandes de formations, stocke localement dans localStorage sous la cle tz_purchases_history.

- Affichage des articles : liste des formations/cours achetes par commande avec prix unitaire
- Statuts de commande : En attente / Accepte / Refuse avec color-coding
- Actions admin : Accepter / Refuser depuis la vue admin sans appel API (mise a jour locale)
- Purge SuperAdmin : bouton Vider permettant de supprimer tout l'historique localStorage

4.3.6. Onglet Historique — Decisions

L'onglet Historique centralise toutes les decisions prises sur les reservations et inscriptions, avec un systeme de filtres combinables.

- Filtre par type : Tous / Reservations / Inscriptions
- Filtre par statut : Tous / Acceptes / Refuses
- Tri chronologique : par date created_at decroissante — plus recent en premier
- Merge des listes : historyReservations + historyInscriptions taguees par _kind pour le rendu

4.3.7. Onglet Statistiques — KPIs et Graphiques

L'onglet Statistiques propose des visualisations CSS pures sans bibliotheque externe, offrant un apercu synthetique de l'activite de la plateforme.

Visualisation	Donnees presentees	Technique CSS
Bar chart — Roles	Nombre de users / admins / superadmins	width: % calcule dynamiquement
Donut chart — Statuts	Actifs / Inactifs / Suspendus en %	conic-gradient CSS dynamique
Bar chart — Activite	KPIs plateforme globaux	width: % sur max normalise
Tableau derniers inscrits	8 derniers utilisateurs par date	Tri .sort() + .slice(0,8)

Technique avancee : le donut chart est genere par calcul des degres proportionnels pour chaque segment, composant dynamiquement la valeur CSS conic-gradient sans aucune dependance externe (Chart.js, D3...).

4.3.8. *Differenciation Admin / SuperAdmin*

Le dashboard adapte visuellement et fonctionnellement son interface selon que l'utilisateur est admin ou superadmin.

Aspect	Admin	SuperAdmin
Theme couleur navbar	Bleu (#1a6fc4)	Violet/Purple (#7c3aed)
Badge de role	Bleu standard	Degradé violet/rose
Suppression utilisateur	Non disponible	Disponible
Changement de role	Non disponible	Disponible
Purge historique achats	Non disponible	Bouton Vider affiche

4.4. Implementation des flux critiques

Cette section detaille l'implementation technique des flux fonctionnels les plus complexes de la plateforme, en analysant les interactions frontend/backend, la gestion des etats React et les strategies de robustesse mises en place.

4.4.1. Flux d'authentification et gestion des roles

L'authentification est basee sur JWT. Apres connexion reussie, le token est stocke dans localStorage et inclus dans chaque requete via les headers HTTP.

Implementation des headers d'authentification

```
const authHeaders = () => ({  
  "Content-Type": "application/json",  
  Authorization: `Bearer ${localStorage.getItem("token")}`,  
});
```

Cycle de vie de la session

- Connexion : stockage du token JWT + objet user dans le state React parent
- Navigation protegee : DashboardRouter verifie user avant de rendre tout composant dashboard
- Deconnexion : setUser(null) + navigate('/login') — le token localStorage est invalide
- Autorisation dynamique : chaque composant verifie user.role pour afficher/masquer les fonctionnalites

4.4.2. Flux de reservation coworking — de l'utilisateur a l'admin

Le flux de reservation implique plusieurs composants et suit un cycle complet de validation administrative.

Etapas du flux

- Etape 1 — Decouverte : l'utilisateur navigue dans l'onglet Salles ou Tables du composant Coworking
- Etape 2 — Modale : clic sur 'Reserver' -> ouverture de la modale avec selection date/heure/tables

- Etape 3 — Soumission : POST /api/coworking/reservations avec statut 'en_attente'
- Etape 4 — Attente : la reservation apparait dans 'Mes Reservations' avec badge en attente
- Etape 5 — Action admin : l'admin voit la demande dans l'onglet Reservations du dashboard
- Etape 6 — Decision : PUT /api/coworking/reservations/:id/statut -> 'acceptee' ou 'refusee'
- Etape 7 — Migration : la reservation quitte l'onglet actif et rejoint l'Historique

Contrainte UX importante : l'onglet 'Mes Reservations' est masque pour les administrateurs. Les admins gerent les reservations exclusivement depuis le Dashboard Admin.

4.4.3. Flux d'inscription aux formations — Panier et validation

Phase 1 — Ajout au panier

- Deux modes d'ajout : formation complete (toggleCart) ou cours individuel (toggleCourseCart)
- Cle de deduplication : id + _type pour les formations, _cartKey (course-{formId}-{courseId}) pour les cours
- Persistance : localStorage.setItem(cartKey, JSON.stringify(cart)) a chaque modification
- Synchronisation cross-page : window.addEventListener('storage', ...) pour les changements depuis d'autres onglets

Phase 2 — Checkout (handleCheckout)

```
POST /api/formations/inscrire-multiple
Body: { formationIds: [1, 3, 7] }
POST /api/formations/:formationId/cours/:courseId/inscrire
```

- Parallelisation : Promise.allSettled() pour traiter simultanement toutes les inscriptions
- Gestion des doublons : statut HTTP 409 -> message 'deja inscrit' sans bloquer les autres inscriptions
- Post-checkout : vidage du panier + localStorage.removeItem() + notification succes

Phase 3 — Validation admin

- Les inscriptions apparaissent dans l'onglet Inscriptions du Dashboard en statut 'en_attente'
- L'admin accepte ou refuse via PUT /api/formations/inscriptions/:id/statut

- Apres traitement, l'inscription migre dans l'Historique avec son statut definitif

4.4.4. Gestion des notifications et feedback utilisateur

La plateforme utilise un systeme de toast notifications personnalise, evitant toute dependance a des bibliotheques tierces.

Implementation dans AdminDashboard

```
const showToast = (msg, type = 'success') => {
  setToast({ msg, type });
  setTimeout(() => setToast(null), 3500);
};
```

- Types supportes : success (vert) et error (rouge)
- Duree d'affichage : 3500ms — auto-disparition via setTimeout
- Positionnement : overlay fixe — visible quelle que soit la position de scroll
- Messages contextuels : differents selon l'action (creation, modification, acceptation, refus, suppression)

4.4.5. Gestion d'etat global et flux de donnees

La gestion d'etat de l'application repose sur les hooks React natifs, sans necessiter Redux ou Context API.

Etat	Hook/Mecanisme	Composant(s) concerne(s)	Persistence
Utilisateur connecte	useState (root App)	Tous via props	Session React
Panier formations	useState + localStorage	Formation, FormationDetail	localStorage
Historique achats	useState + localStorage	AdminDashboard (Achats)	localStorage
Token JWT	localStorage	authHeaders() — tous	localStorage
Donnees API	useState + useCallback	Composants individuels	State React
Onglet actif dashboard	useState (activeTab)	AdminDashboard	Session React

4.4.6. Modaux et formulaires CRUD

Les operations de creation et modification d'entites utilisent un composant Modal generique reutilise dans tous les contextes.

- Composant Modal generique : wrapper { title, onClose, children } avec overlay et fermeture
- Etat de formulaire : objet form gere par useState avec mise a jour partielle via setForm
- Validation inline : verification des champs obligatoires avant l'appel API avec toast d'erreur
- Confirmation suppression : modal de confirmation distincte avec message d'alerte irreversibilite

Synthese du Chapitre 4

Ce chapitre a detaille la realisation technique complete de la plateforme TZ Prime Solutions, depuis le choix de la stack jusqu'a l'implementation des flux fonctionnels critiques.

La realisation technique de la plateforme TZ Prime Solutions illustre une approche moderne et pragmatique du developpement web full-stack :

- Architecture SPA React avec routage cote client et separation nette des composants par domaine fonctionnel
- Securite par JWT avec controle des acces granulaire (user / admin / superadmin) a tous les niveaux
- Flux de reservation et d'inscription complets avec cycle de vie en_attente -> acceptee/refusee
- Dashboard d'administration riche en fonctionnalites, differencie visuellement selon le niveau d'accès
- Systeme de panier persistant cross-composant via localStorage et synchronisation par evenements natifs
- Visualisations statistiques CSS pures, sans dependance externe, maintenant la legerete du bundle

L'ensemble de ces choix techniques reflète une priorité accordée à la maintenabilité, la séparation des préoccupations et l'expérience utilisateur, posant des bases solides pour l'évolution future de la plateforme.

Chapitre 5

Documentation et tests des API

5.1. Introduction

Après avoir présenté, dans le chapitre précédent, la réalisation technique de la plateforme TZ Prime Solutions, ce cinquième chapitre est consacré à la documentation et à la validation de l'API REST développée côté serveur. Nous présenterons dans un premier temps l'ensemble des points d'accès (endpoints) exposés par le Backend, organisés par module fonctionnel, ainsi que les niveaux d'accès requis pour chacun d'eux. Nous détaillerons ensuite l'outil utilisé pour tester ces endpoints, avant d'illustrer concrètement deux scénarios de test représentatifs : la création d'un compte utilisateur et la consultation des statistiques administratives. Nous clôturerons ce chapitre par une synthèse des mécanismes de sécurité mis en place au niveau de l'API.

5.2 Outils et environnement de test

Les tests des endpoints de l'API ont été réalisés à l'aide de Postman, un client HTTP graphique largement utilisé pour le développement et la validation d'API REST. Cet outil permet de composer des requêtes HTTP (GET, POST, PUT, DELETE), de configurer les en-têtes et le corps de la requête, de gérer l'authentification via un jeton Bearer, et d'inspecter en détail la réponse renvoyée par le serveur (statut HTTP, temps de réponse, corps JSON).

Le serveur Backend a été lancé localement sur le port 5000 (<http://localhost:5000>), exposé via Express.js. L'ensemble des routes de l'API est préfixé par /api, puis organisé par module : /api/auth pour l'authentification, /api/admin pour l'administration, /api/coworking pour la gestion des espaces de coworking, et /api/formations pour la gestion des formations et des inscriptions.

5.3 Documentation des endpoints de l'API

L'API REST de TZ Prime Solutions expose au total une quarantaine d'endpoints répartis sur quatre routeurs Express distincts. Le tableau 5.1 récapitule l'ensemble de ces points d'accès, leur méthode HTTP, le niveau

d'accès requis (Public, Utilisateur authentifié, Administrateur ou Super administrateur) ainsi qu'une description fonctionnelle succincte.

5.3.1. Module Authentication (/api/auth)

Ce module gère l'inscription, la connexion et la récupération du profil de l'utilisateur connecté. Les routes /register et /login sont protégées par un rate limiter (20 tentatives par tranche de 15 minutes) afin de prévenir les attaques par force brute. Le mot de passe est haché avec bcrypt (coût 12) et un hachage factice (DUMMY_HASH) est comparé en cas d'utilisateur inexistant, afin d'éviter les attaques par mesure du temps de réponse (timing attack).

Tableau 5.1. Endpoints du module Authentication (/api/auth)

Méthode	Endpoint	Accès	Description
POST	/api/auth/register	Public	Créer un nouveau compte utilisateur (rôle 'user' par défaut)
POST	/api/auth/login	Public	Authentifier un utilisateur et générer un jeton JWT
GET	/api/auth/me	Utilisateur	Récupérer le profil de l'utilisateur connecté

5.3.2. Module Administration (/api/admin)

Ce module regroupe les fonctionnalités réservées aux administrateurs et super administrateurs : gestion des utilisateurs (consultation, changement de statut, changement de rôle, suppression) ainsi que la consultation des statistiques globales de la plateforme (nombre d'utilisateurs, de formations, de salles, de tables, de réservations et d'inscriptions). Les opérations sensibles, telles que le changement de rôle ou la suppression d'un compte, sont réservées au super administrateur et incluent des garde-fous empêchant la suppression ou la rétrogradation du dernier super administrateur, ainsi que la modification de son propre compte.

Tableau 5.2. Endpoints du module Administration (/api/admin)

Méthode	Endpoint	Accès	Description
GET	/api/admin/users	Admin	Lister tous les utilisateurs de la plateforme
PUT	/api/admin/users/:id/statut	Admin	Changer le statut d'un utilisateur (actif/inactif/suspendu)
PUT	/api/admin/users/:id/role	Super admin	Changer le rôle d'un utilisateur (user/admin/superadmin)
DELETE	/api/admin/users/:id	Super admin	Supprimer définitivement un utilisateur
GET	/api/admin/stats	Admin	Récupérer les statistiques globales de la plateforme

5.3.3. Module Coworking (/api/coworking)

Ce module, le plus volumineux de l'API avec dix-sept endpoints, gère l'ensemble du cycle de vie des ressources de coworking : les salles, les emplacements, les tables, ainsi que les réservations associées. Les opérations de création, modification et suppression des ressources (salles, emplacements, tables) sont réservées aux administrateurs, tandis que la consultation est ouverte à tout utilisateur authentifié. Le flux de réservation suit un cycle de vie en trois états (en_attente, acceptée, refusée) : une table ou une salle n'est effectivement bloquée qu'après acceptation explicite de la réservation par un administrateur, via une transaction SQL garantissant la cohérence entre la mise à jour du statut de la réservation et la disponibilité de la ressource associée.

Tableau 5.3. Endpoints du module Coworking (/api/coworking)

Méthode	Endpoint	Accès	Description
GET	/api/coworking/salles	Utilisateur	Lister toutes les salles de coworking
POST	/api/coworking/salles	Admin	Créer une nouvelle salle
PUT	/api/coworking/salles/:id	Admin	Modifier une salle existante
DELETE	/api/coworking/salles/:id	Admin	Supprimer une salle
GET	/api/coworking/emplacements	Utilisateur	Lister tous les emplacements (avec nom de la salle)

POST	/api/coworking/emplacements	Admin	Créer un nouvel emplacement
PUT	/api/coworking/emplacements/:id	Admin	Modifier un emplacement existant
DELETE	/api/coworking/emplacements/:id	Admin	Supprimer un emplacement
GET	/api/coworking/tables	Utilisateur	Lister toutes les tables (avec salle/emplacement)
POST	/api/coworking/tables	Admin	Créer une nouvelle table
PUT	/api/coworking/tables/:id	Admin	Modifier une table existante
DELETE	/api/coworking/tables/:id	Admin	Supprimer une table
POST	/api/coworking/reservations	Utilisateur	Créer une réservation (salle ou table)
GET	/api/coworking/reservations/me	Utilisateur	Lister ses propres réservations
GET	/api/coworking/reservations	Admin	Lister toutes les réservations de la plateforme
PUT	/api/coworking/reservations/:id/statut	Admin	Accepter ou refuser une réservation
DELETE	/api/coworking/reservations/:id	Utilisateur/Admin	Annuler une réservation (propriétaire ou admin)

5.3.4. Module Formations (/api/formations)

Ce module gère le catalogue des formations ainsi que les inscriptions des utilisateurs. Il propose, en complément des opérations CRUD classiques sur les formations (réservées aux administrateurs), une route spécifique d'inscription multiple (/inscrire-multiple) permettant à un utilisateur de s'inscrire simultanément à plusieurs formations en une seule requête, avec gestion transactionnelle des doublons (une formation déjà suivie n'est pas re-inscrite et est signalée dans la réponse). Un soin particulier a été apporté à l'ordre de déclaration des routes Express : les routes statiques (/inscriptions/me, /inscrire-multiple, etc.) sont déclarées avant les routes paramétrées (/:id) afin d'éviter tout conflit de routage.

Tableau 5.4. Endpoints du module Formations (/api/formations)

Méthode	Endpoint	Accès	Description
GET	/api/formations/inscriptions/me	Utilisateur	Lister ses propres inscriptions aux

			formations
GET	/api/formations/inscriptions/all	Admin	Lister toutes les inscriptions de la plateforme
PUT	/api/formations/inscriptions/:id/statut	Admin	Changer le statut d'une inscription
DELETE	/api/formations/inscriptions/:id	Admin	Supprimer une inscription
POST	/api/formations/inscrire-multiple	Utilisateur	S'inscrire à plusieurs formations en une seule requête
GET	/api/formations	Utilisateur	Lister toutes les formations disponibles
POST	/api/formations	Admin	Créer une nouvelle formation
GET	/api/formations/:id	Utilisateur	Récupérer le détail d'une formation
PUT	/api/formations/:id	Admin	Modifier une formation existante
DELETE	/api/formations/:id	Admin	Supprimer une formation
POST	/api/formations/:id/inscrire	Utilisateur	S'inscrire à une formation donnée
DELETE	/api/formations/:id/desinscrire	Utilisateur	Se désinscrire d'une formation

5.4 Tests fonctionnels illustres

Afin d'illustrer concrètement le fonctionnement de l'API, deux scénarios de test représentatifs sont présentés ci-dessous : la création d'un compte utilisateur via le module d'authentification, et la consultation des statistiques administratives via une route protégée par jeton JWT.

5.4.1. Test de création de compte (POST /api/auth/register)

La requête POST est envoyée à l'URL `http://localhost:5000/api/auth/register` avec un corps JSON contenant le nom, l'email et le mot de passe du nouvel utilisateur. Le serveur répond avec un statut 201 Created, accompagné d'un message de confirmation, du jeton JWT généré et des informations de l'utilisateur créé (avec le rôle 'user' attribué par défaut). La figure 5.1 illustre ce test réalisé sous Postman.

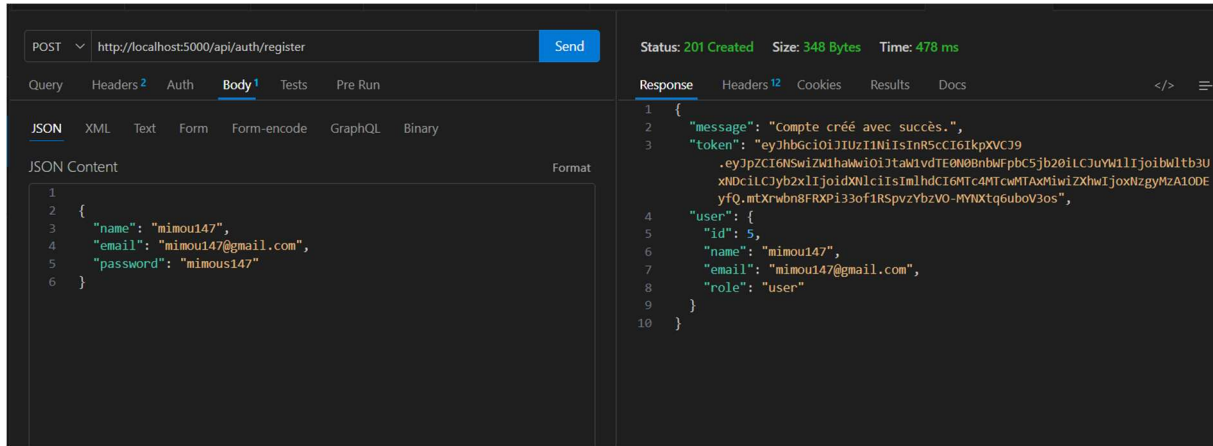


Figure 5.1. Test Postman de la route POST /api/auth/register (statut 201 Created)

5.4.2. Test d'accès aux statistiques administratives (GET /api/admin/stats)

La requête GET est envoyée à l'URL <http://localhost:5000/api/admin/stats> avec un en-tête d'autorisation de type Bearer Token, contenant le jeton JWT obtenu lors de la connexion d'un compte administrateur. Le middleware `authMiddleware` vérifie la validité du jeton, puis `adminMiddleware` vérifie que le rôle de l'utilisateur autorise l'accès à cette route. Le serveur répond avec un statut 200 OK et un objet JSON contenant les compteurs globaux (utilisateurs, formations, salles, tables, réservations, inscriptions) ainsi que les cinq dernières réservations et inscriptions enregistrées. La figure 5.2 illustre ce test.

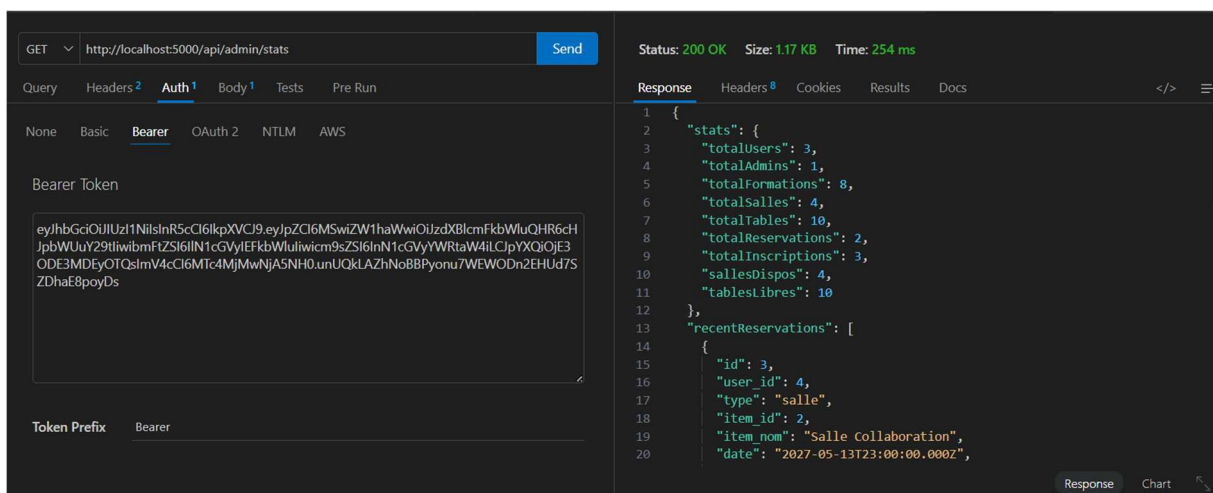


Figure 5.2. Test Postman de la route GET /api/admin/stats (statut 200 OK)

5.5 Securite de l'API

La conception de l'API repose sur plusieurs mecanismes de securite complementaires, valides au cours des tests presentes precedemment :

- Authentification par jeton JWT : chaque requete vers une route protegee doit inclure un en-tete Authorization de type Bearer Token, verifie par le middleware authMiddleware
- Controle d'accès basé sur les rôles (RBAC) : trois niveaux distincts (user, admin, superadmin) sont appliques via des middlewares dedies (adminMiddleware, superAdminMiddleware) sur chaque route sensible
- Hachage des mots de passe avec bcrypt (cout 12), garantissant qu'aucun mot de passe n'est jamais stocke ou transmis en clair
- Limitation du taux de requetes (rate limiting) sur les routes /register et /login, fixee a 20 tentatives par tranche de 15 minutes, afin de prevenir les attaques par force brute
- Protection contre les attaques par mesure du temps de reponse (timing attack) lors de la connexion, grace a l'utilisation d'un hachage factice (DUMMY_HASH) compare systematiquement, meme lorsque l'utilisateur n'existe pas
- Garde-fous métier au niveau des routes d'administration, empechant la suppression ou la retrogradation du dernier super administrateur, ainsi que la modification de son propre compte
- Validation systematique des entrees (champs obligatoires, format d'email, longueur du mot de passe, valeurs autorisees pour les statuts et roles) avant tout traitement ou acces a la base de donnees

Ce chapitre a presente la documentation complete des endpoints de l'API REST de TZ Prime Solutions, ainsi que leur validation a travers des tests fonctionnels realises sous Postman.

Conclusion

Ce chapitre a permis de documenter de maniere exhaustive l'ensemble des points d'accès exposes par l'API REST de TZ Prime Solutions, organisees en quatre modules fonctionnels totalisant une quarantaine d'endpoints. Les tests realises sous Postman ont permis de valider concretement le bon fonctionnement de deux flux representatifs : la creation d'un compte utilisateur et la consultation securisee des statistiques administratives. Enfin, la synthese des mecanismes de securite mis en place — authentication JWT, controle d'accès par rôle, hachage des mots de passe et limitation du taux de requetes — confirme la robustesse de l'architecture Backend

face aux principales menaces visant les API web. Ces elements de validation viennent ainsi completer la realisation technique presentee au chapitre precedent, et attestent de la fiabilite de la plateforme avant sa mise en production.

Conclusion Générale

Ce projet de fin d'études nous a permis de concevoir et de développer TZ Prime Solutions, une plateforme web unifiée dédiée à la gestion des espaces de coworking et des programmes de formation au sein de la société Logicom. Tout au long de ce travail, nous avons suivi une démarche structurée, allant de l'analyse du contexte et des besoins jusqu'à la réalisation technique complète de la solution.

Le premier chapitre nous a permis de présenter la société d'accueil ainsi que le contexte et la problématique à l'origine de ce projet, en mettant en évidence l'absence, sur le marché actuel, d'une solution combinant à la fois la gestion du coworking et celle de la formation. Le deuxième chapitre a été consacré à l'analyse et à la spécification des besoins fonctionnels et non-fonctionnels, à travers l'identification des différents acteurs et la définition précise des exigences de chacun des deux modules de la plateforme. Le troisième chapitre a porté sur la conception de la solution, en détaillant l'architecture technique multicouches retenue ainsi que la modélisation de la base de données à partir du diagramme de classes UML. Le quatrième chapitre a présenté la réalisation technique de la plateforme, en illustrant les choix technologiques adoptés et l'implémentation des principales fonctionnalités du Frontend et du Dashboard d'administration. Enfin, le cinquième chapitre a documenté l'ensemble des endpoints de l'API REST développée et a illustré leur validation à travers des tests fonctionnels réalisés sous Postman, confirmant la fiabilité et la sécurité de la solution mise en place.

Sur le plan technique, l'utilisation conjointe de React pour le Frontend et de Node.js pour le Backend nous a permis de mettre en place une architecture moderne, modulaire et évolutive, capable de répondre efficacement aux besoins identifiés. La séparation claire des responsabilités entre les différentes couches de l'application a par ailleurs facilité la maintenabilité et l'évolutivité future de la plateforme.

Sur le plan personnel, ce projet a été l'occasion de mettre en pratique l'ensemble des connaissances acquises durant notre formation, tout en développant de nouvelles compétences, tant techniques que méthodologiques, notamment à travers l'adoption de la méthodologie agile Scrum pour le pilotage du projet.

Comme perspectives, plusieurs pistes d'amélioration peuvent être envisagées pour enrichir la plateforme TZ Prime Solutions, telles que l'intégration d'un système de paiement en ligne complet, le développement d'une application mobile dédiée, ou encore l'ajout de fonctionnalités d'intelligence artificielle pour la recommandation personnalisée de formations et d'espaces de travail adaptés aux besoins de chaque utilisateur.